SP-117 PBI Customer - Final Report

Section 01, Spring Semester, 2026
Team Members: John Roehsler and Michael Orusa
Instructor: Sharon Perry
Date: 04/20/2026
Website: https://powerbiseniorproject.github.io/SP-117-PBI-Customer/
GitHub: https://github.com/PowerBISeniorProject/SP-117-PBI-Customer
NDA Status: No NDA
Stats and Status:

Lines of Code (LOC)	142
Components	vaderSentiment, pandas, Python, GitHub
Hours Estimate	396
Hours Actual	295
Status	100% Complete

Table of Contents

Introduction

This project focuses on the design and implementation of a data analytics pipeline and Power BI dashboard that integrates structured customer purchase data with unstructured customer review text to support customer and product level insight generation. In modern business environments, companies rely heavily on data driven decision making, and tools that can transform large datasets into understandable visual insights are critical. Customer reviews contain valuable feedback that often exists in text form, making it difficult to analyze without additional processing. By combining structured product data with text based sentiment results, this system provides a more complete view of customer behavior and product performance.

The system takes in transactional data and review datasets, applies text preprocessing and sentiment analysis using a Python based natural language processing approach, and reintegrates sentiment scores into the analytical model. The sentiment scoring process allows written customer opinions to be translated into measurable values that can be compared alongside numerical ratings and pricing information. Once processed, the data is modeled using a star schema to improve organization and performance, and it is visualized in Power BI through interactive dashboards that allow users to explore trends by product, category, pricing level, and time period.

The scope of this project includes multiple stages of development, including data acquisition, data cleaning, sentiment scoring, data modeling, DAX measure development, and validation of both sentiment classification and visual outputs. Each stage helps build a reliable pipeline that transforms raw data into meaningful insights. The dashboard was designed with usability in mind so that both technical and non technical users can easily understand the results through clear visuals and interactive filtering tools.

The overall goal of this project is to demonstrate how combining structured and unstructured data can improve business analysis and support better decision making. By using industry relevant tools such as Python and Power BI, the project reflects common real world analytics workflows. The final system provides an example of how customer feedback data can be processed, analyzed, and visualized to identify patterns, evaluate product performance, and better understand customer sentiment.

Challenges, Solutions, and Assumptions

Challenges & Solutions:

- Power BI .pbix files do not work with Git
    - Dashboard is stored, updated, and managed on one device.
- Currency conversion (Indian Rupees to USD)
    - Clean.py converts Indian Rupees to USD using an accurate conversion rate.
- Review titles being too short/noisy for reliable sentiment
    - Sentiment.py scores both the title and content and then averages them to get reliable sentiment results.
- Inconsistent data types
    - Special characters like '₹', '%', and ',' are converted to numeric types.

Assumptions:

- The project assumes that the selected Kaggle Amazon dataset contains sufficient and accurate information to support product analysis. It is assumed that the dataset remains freely available for academic use and represents real customer review data. Additionally, the project assumes that the dataset will be static and that any data quality issues can be addressed through preprocessing and cleaning prior to analysis.

Requirements

Overview:

- This project delivers a customer insights dashboard developed using Power BI. The system integrates structured product and transaction information with unstructured customer review text. Review text will be cleaned and scored for sentiment using VADER. The resulting sentiment metrics will be integrated into the dataset and modeled in Power BI using a star schema. The final dashboard will provide interactive visualizations that allow users to explore product, brand, pricing, and time-based performance trends.
- The primary goal of the system is to support business analysis by answering questions such as identifying top performing and lowest performing products and brands, and examining the impact of pricing and discounts on customer sentiment. An additional goal is to evaluate the effectiveness of sentiment analysis by comparing VADER sentiment scores with the star ratings provided in customer reviews.

Functional Requirements :

- Data Import & Preparation
    - The system shall import the chosen Kaggle Amazon dataset into a staging area.
    - The system shall remove columns that are not relevant to customer/product/brand analysis.
    - The system shall standardize key fields (dates, prices, discount %, rating) into usable data types.
    - The system shall convert currency fields to a consistent currency format for reporting (USD), including labeling assumptions used in conversion.
- Sentiment Scoring Pipeline
    - The system shall preprocess review text prior to sentiment scoring.
    - The system shall generate sentiment scores using VADER.
    - The system shall classify each review into sentiment labels (positive/neutral/negative) using defined thresholds.
    - The system shall output a processed dataset that merges sentiment results back to each review record.
- Data Modeling
    - The system shall create a data model organized as a star schema where feasible.
    - The system shall create relationships between review records, product attributes, and time attributes.
    - The system shall include calculated measures (DAX) for metrics such as average rating, average sentiment, review count, and sentiment distribution.
- Dashboard Insights & Visuals
    - The dashboard shall display top-performing and lowest-performing products based on rating and/or sentiment.
    - The dashboard shall visualize sentiment trends over time and/or across categories.

    - The dashboard shall support analysis of correlation/relationships between price, discount, and sentiment.
    - The dashboard shall provide detailed views that help explain why products/brands are high or low performing.
- Interactivity Requirements
    - The dashboard shall include slicers/filters for product, brand, category, and time period.
    - The dashboard shall support “drill-down” or “drill-through” from summary visuals to detail pages.
- Validation Requirements
    - The dashboard shall include at least one validation visualization (sentiment score vs rating scatter, agreement rate by rating bucket).
    - The system shall compare sentiment outputs against the review rating to evaluate alignment.

Non-Functional Requirements

- Security
    - A public dataset will be used and any remaining personal information will be removed and never processed.
- Capacity
    - The dashboard will have ample performance and will be able to run on a standard desktop or laptop capable of running Power BI Desktop edition.
- Usability
    - The dashboard’s visuals shall be understandable for a non-technical user. Clear titles, labels, and consistent formatting will be implemented.
- Other
    - The sentiment scoring and text preprocessing steps shall be reproducible using a Python script. This script, along with supporting documentation, will be stored and version controlled in GitHub to ensure transparency and repeatability of results.

Design

Overview

- The system is designed as a small analytics pipeline that produces a Power BI dashboard. Data flows from the Kaggle Amazon dataset through cleaning, then through a Python sentiment scoring module using VADER, and finally into a modeled Power BI dataset used for reporting and visualization.

Architectural Strategy

- VADER:
    - VADER is a rule/lexicon-based sentiment approach that performs well on short, opinionated text like reviews and does not require training a model. It is fast and easy to explain and works well with how many reviews have emojis and other non-text based characters.
- Power BI:
    - A common industry tool, strong for interactivity with visuals, works well with tabular modeling and DAX, and is what was offered to use to complete the project.
- Star Schema:
    - Improves filter performance and makes measures/relationships clearer

System Architecture

The system is organized as a pipeline with the following components. Data flows from the data source through preparation and sentiment scoring into the Power BI semantic model and finally into the visualization layer.

- Component A: Data Source Layer
    - Kaggle Amazon reviews dataset
- Component B: Data Preparation Layer
    - Clean the dataset by removing unused columns, handling missing values, and converting currencies.
- Component C: Sentiment Scoring Layer
    - Generate sentiment scores from review text using VADER.
- Component D: Power BI Modeling Layer
    - Import the processed dataset and create dimensions (Product, Brand, Date, and Category), relationships, and DAX measures. Page 3 of 7
- Component E: Visualization Layer
    - Generate report pages showcasing overview, product performance, brand performance, sentiment vs rating validation, discount analysis, and drill through detail pages.

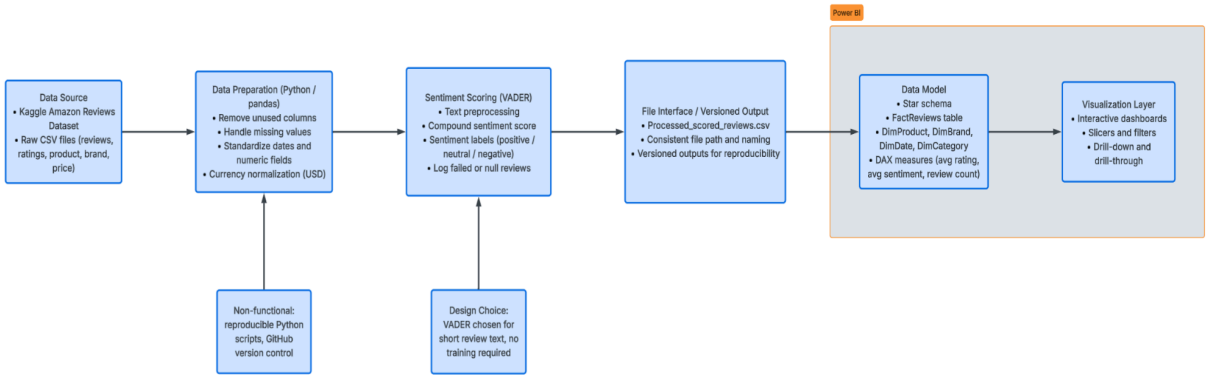

Figure 1: System Architecture

Detailed Module Design

- Module 1: Data Cleaning & Transformation
  - Classification: Data preparation
  - Definition: Loads raw Kaggle dataset and produces a cleaned standardized dataset that will be used for sentiment scoring and modeling.
  - Inputs: Raw dataset file
  - Outputs: Cleaned dataset with standardized columns
  - Responsibilities: Clean the dataset by removing unused columns, handling missing values, and converting currencies
  - Constraints: Cleaning steps must be reproducible and documented and do not remove data that would skew sentiment scoring results
  - Error Handling: Log invalid numeric conversions
  - Interface/Exports: Output file is used by Sentiment Scoring Script
- Module 2: Sentiment Scoring Script
  - Classification: Python script
  - Definition: Takes in a cleaned dataset, computes sentiment, and writes processed files.
  - Inputs: Cleaned dataset containing review text and rating
  - Outputs: Processed dataset containing sentiment fields
  - Responsibilities: Preprocessing, scoring, labeling, and exporting
  - Constraints: Handle null text or non english review text
  - Error Handling: Rows that fail are logged and assigned a null value to not skew statistics
  - Interface/Exports: Output file is used by Power BI
- Module 3: File Interface
  - Classification: File based data exchange
  - Definition: Controls how datasets move between Python and Power BI
  - Inputs: Scored dataset file
  - Outputs: Versioned processed dataset used by Power BI refresh
  - Responsibilities: Maintain consistent filenames and paths so Power BI does not break. Maintain version outputs.
  - Constraints: Stored locally

    - Error Handling: If a file is not found an error is logged and the system reverts back to the last known working file.
    - Interface/Exports: Output file is used by Power BI
- Module 4: Power BI Data Model
    - Classification: Power BI model
    - Definition: Star schema with relationships and DAX measures
    - Inputs: Scored dataset file
    - Outputs: Power BI model with tables, relationships, and measures
    - Responsibilities: Enable filtering, aggregations, and performance metrics
    - Constraints: Relationship keys must be unique in dimensions
    - Error Handling: Validate relationships and fix duplicates
- Module 5: Dashboard Pages
    - Classification: UI and report pages
    - Definition: Visuals answering target questions
    - Inputs: Power BI semantic model
    - Outputs: Report pages and visuals
    - Responsibilities: Interactivity, drill through, and consistent formatting
    - Constraints: Must be easy to understand and read for non technical users
    - Error Handling: Validate visuals to respond to filters correctly and verify there are no broken fields after a refresh
    - Interface/Exports: Power BI report file

Design Assumptions and Constraints

- Assumptions:
    - The dataset contains review text and product ratings and includes product/brand fields usable for grouping. Python libraries required include VADER and standard data tools, like pandas. Power BI Desktop is the primary reporting environment.
- Constraints:
    - Power BI files are difficult to merge. To address this, the team will control dashboard file edits using a shared environment/workflow, or device if VM does not provide sufficient support. Supporting scripts and processed outputs are tracked in GitHub.

Development

System Integration & Workflow Overview

- This project was built using Microsoft Power BI to analyze Amazon product review data and turn it into an interactive dashboard. The main idea was to take a raw dataset and turn it into visuals that clearly show trends in ratings, sentiment, categories, and pricing.
- All components connect through the same dataset. For example, the category slicer controls the entire dashboard. When a category is selected, every visual updates automatically to show only the relevant data. This keeps everything synchronized and makes the dashboard interactive instead of static.

Component descriptions

- Average Rating KPI Card
    - This component shows the overall average rating across all products. It provides a quick summary of customer satisfaction without needing to analyze individual products. It uses the rating field and calculates the average rating across the dataset.
    - The KPI card was created by placing the rating field into a card visual and setting the aggregation to average. Formatting changes were applied to highlight the number using orange text so it stands out as the main summary value. Testing confirmed that the value updates correctly when categories are filtered.
- Category Slicer
    - Category slicer allows users to filter dashboards by product category. This makes the dashboard interactive and allows users to focus on specific groups of products. It uses the category field to filter all connected visuals.
    - The slicer was created by inserting a slicer visual and assigning the category field. Single selection mode was enabled to keep filtering simple. Formatting changes were made to match the dark theme and highlight selected categories using orange.
- Review Sentiment Distribution
    - This chart shows how many reviews fall into each sentiment group (Positive, Negative, Neutral). It helps visualize the overall tone of customer feedback. It uses sentiment (category field) and count of records.
    - A column chart was created using the sentiment field on the X-axis and the count of records on the Y-axis. Colors and labels were formatted to match the dashboard theme. Testing confirmed that the chart updates correctly when filtering by category.
- Average Rating by Category
    - This chart compares average ratings across product categories. It helps identify which categories perform better or worse. It uses category (X-axis)and average rating (Y-axis).

    - A bar chart was created using category as the grouping field and rating as the measured value. The aggregation was set to average. Labels were formatted for readability and tested to confirm correct updates during filtering.
- Top 10 Products by Rating
    - This chart displays the highest-rated products in the dataset. It helps identify top-performing products that customers rate highly. It uses: product_name, average rating, and Top 10 filter.
    - A horizontal bar chart was created using product_name and rating. A Top 10 filter was applied based on average rating. Labels were shortened automatically to improve readability. Testing confirmed that the list updates dynamically with slicer selections.
- Price vs Rating Scatter Plot
    - This scatter plot shows the relationship between product price and rating. It helps determine whether higher-priced products receive better ratings. It uses discounted_price_usd (X-axis) and rating (Y-axis)
    - A scatter plot was created using discounted price and rating fields. Gridlines were added to improve readability. Formatting adjustments were made to ensure consistency with the overall theme. Testing confirmed correct behavior during category filtering.

Global Design Decisions

- Power BI was chosen because it supports interactive filtering and multiple visual types in one place. Aggregations like averages and counts were used across visuals to summarize large amounts of data into readable insights.
- A consistent dark theme with orange and blue accents was applied across the dashboard. This improved readability and created a professional visual style. The design also focused on keeping the layout simple so users can easily understand the information being displayed.

Database Connections & External Connections

- Connection Overview:
    - Not Applicable. The primary data source is a CSV file that was sourced from Kaggle. It contains product reviews, ratings, and pricing information.
- Configuration Details:
    - The system relies on local file exchanges rather than databases or APIs. The main configuration requirement is having consistent file paths and naming conventions so that the Power BI data refresh process doesn't break. Python scripts are managed and stored in GitHub.
- Integration Logic:
    - Cleaning: Python module loads raw CSV file from Kaggle and cleans the data by removing unneeded columns or adjusting fields.
    - Sentiment Processing: The VADER module in Python is used to calculate sentiment scores.

    - Data Modeling: Power BI is the final integration point where the cleaned data and sentiment scores are ingested and visualized using a star schema data model.
    - Error Handling: If Power BI cannot find needed files, then the system logs an error and will use a previously saved file.

Project Setup

- Prerequisites
    - Hardware: A standard desktop or laptop running Windows 10 or 11.
    - Microsoft Power BI Desktop: Required for data modeling and visualization.
    - Python Environment: Required for executing the data cleaning and sentiment scoring scripts.
    - Python Libraries: Pandas for data manipulation and vaderSentiment for sentiment scoring.
    - Version Control: Git installed and access to GitHub for pulling scripts and data.
- Installation Steps:
    - Clone the Repository: Do a git pull of the GitHub repository on your local machine to access the scripts and CSV file.
    - Prepare Python Environment: Ensure you have a Python environment and the required libraries (pandas and VADER).
    - Clean the Data: Execute the cleaning scripts to remove unused columns and handle field adjustment.
    - Sentiment Scoring: Run the sentiment scoring script to process the cleaned data and generate sentiment scores.
    - Load Power BI: Open the provided Power BI file in Power BI Desktop and load in the data.
- Verification:
    - Data Pipeline Verification: Navigate to the directories and make sure that cleaned data and sentiment score files are there.
    - Dashboard Verification: Launch the dashboard in Power BI and make sure all the pages and visuals load without error.

Test Plan

Test Case ID	Description	Test Procedures	Expected Result
TC-01	Verify Data Cleaning and Sentiment Scoring Pipeline.	Execute the Python data cleaning script on the raw Kaggle dataset. Run VADER sentiment scoring script on the cleaned data. Open the resulting output .csv file.	The output file is generated successfully, unused columns are removed, currency is normalized to USD, and compound sentiment scores and labels are accurately appended to each review.
TC-02	Verify Power BI Data Modeling and DAX Measures.	Open the provided Power BI .pbix file. Refresh the data source to load the processed_scored_reviews.csv file. Go to Model view to inspect relationships.	Data loads without errors. The star schema is maintained with proper 1-to-many relationships, and DAX measures (Average Rating, Average Sentiment) calculate accurately without throwing syntax errors.
TC-03	Verify Dashboard Visual Renderings	Navigate to the main dashboard report page in Power BI and then navigate through each page.	All defined visual components render successfully and match the specified dark theme with orange accents.
TC-04	Verify Interactivity and Slicer Filtering.	On the main dashboard, locate the Category Slicer. Select a single category. Observe the data changes on the KPI Card	All visuals, KPI cards, and charts on the dashboard update dynamically to reflect only the data from the selected category,

		and all surrounding charts.	with no errors or broken visuals.
TC-05	Verify Sentiment vs. Rating Validation.	Navigate to the validation visualization comparing sentiment scores to customer star ratings.	The visual successfully maps the VADER sentiment labels against the star rating buckets, demonstrating the alignment or discrepancies between text sentiment and numerical rating.

Test Report

Test Case	Tester	Pass/Fail	Severity	Severity Description
TC-01	John	Pass	N/A	N/A
TC-02	John	Pass	N/A	N/A
TC-03	Michael	Fail	Low	Some text is not legible due to the background and text being similar colors.
TC-04	Michael	Pass	N/A	N/A
TC-05	Michael	Fail	Low	The VADER sentiment score vs reviews works as intended, but the graph is hard to read.

- Testing initially identified minor low severity readability issues in two dashboard visuals. These issues did not affect data accuracy or dashboard functionality. After the adjustments were made, all test cases passed and the dashboard met the project requirements.

Version Control

- GitHub was chosen for version control for this project. The GitHub repository contained the Python scripts, documentation, datasets, and other support files for development. A dedicated GitHub account and repository were created for this project. We were not able to use GitHub for the Power BI .pbix files, however. Due to the binary nature of .pbix files, they would not be able to be stored on GitHub for version control. To work around this, we decided to keep the dashboard on one computer for data integrity purposes, which ensured that only one working copy existed at a time and prevented conflicts between team members.

Conclusion

This project successfully delivered a working data analytics pipeline and interactive Power BI dashboard designed to analyze Amazon product review data. The system takes raw customer review and product data, cleans and standardizes it using Python, applies VADER sentiment scoring, and integrates the processed results into a Power BI star schema model. The final dashboard provides clear visual insights into product ratings, sentiment distribution, pricing relationships, and category performance. All major functional requirements were implemented, including sentiment classification, interactive filtering, and validation visuals comparing sentiment scores to customer ratings.

Testing confirmed that the pipeline components worked together as expected and that the dashboard responded correctly to slicers and filtering actions. Minor visual readability issues were identified during testing, but these were considered low severity and did not impact the accuracy of the system output. Overall, the system demonstrates how structured and unstructured data can be combined to support meaningful business analysis and decision making.

Future improvements could include refining dashboard formatting for improved readability, expanding the dataset to include additional time periods or product sources, and experimenting with alternative sentiment analysis models to compare accuracy against VADER. Additional automation of file handling and dataset refresh steps could also improve efficiency. Despite these possible enhancements, the current implementation meets the project goals and provides a functional and reproducible analytics solution that supports customer and product level insight generation.

Appendix

					Milestone #1				*Milestone #2*				*Milestone #3*				C-Day(tentative)	
Phase	**Tasks**	**Complete%**	**Current Status Memo**	**Assigned To**	**01/25**	**02/01**	**02/08**	**02/15**	**02/22**	**03/01**	**03/08**	**03/15**	**03/22**	**03/29**	**04/05**	**04/12**	**04/19**	**04/26**
Requirements	Review project scope outline	100%	Complete	John, Michael	20	12	5											
	Define requirements	100%	Complete	John, Michael														
	Review requirements with Prof.	100%	Complete	John, Michael		10	15											
	Get sign off on requirements	100%	Complete	John, Michael			5	10	4									
Project design	Define tech required	100%	Complete	John				10	4									
	Data model design	100%	Complete	Michael				5	10	10								
	Front-end/dashboard design	100%	Complete	John						10	10							
	Pipeline design	100%	Complete	Michael			6	6	10	5	5							
	Develop working prototype	100%	Complete	John						10	10							
	Test prototype	100%	Complete	Michael							5	10	5					
Development	Acquire datasets	100%	Complete	John								8	5	10				
	Clean structured purchase data	100%	Complete	Michael								8	10	20	20			
	Clean and review text data	100%	Complete	John														
	Build sentiment scoring script	100%	Complete	Michael											10	10		
	Build Power BI measures and v	100%	Complete	John														
	Add interactivity	100%	Complete	Michael														
	Validate and refine dashboard	100%	Complete	John										8	5	20		
Final report	Presentation preparation	100%	Complete	John, Michael												15	10	10
	Poster preparation	100%	Complete	John, Michael														10
	Final report submission to D2L and project owner	100%	Complete	John, Michael														5
			Total work hours	**396**	**20**	**22**	**31**	**31**	**28**	**35**	**30**	**26**	**20**	**38**	**35**	**45**	**10**	**25**

GanttChart Estimate